

World-Model-Anchored Preference Optimization for Joint Flow-Matching Policies

Working draft

Rooholla Khorrambakht
(co-authors TBD)

May 10, 2026

Abstract

We propose *World-Model-Anchored Preference Optimization* (WAPO), a method for aligning a play-pretrained joint world-action transformer with small demonstration sets, without reward modeling and without distilling the policy into a tractable parametric family. Our backbone is a single causal flow-matching denoiser exposing independent per-frame noise levels for state and action, which lets the same network act as a world model, a policy, or a random-action proposer at inference time. We adapt the Diffusion-DPO bound to this dual-channel flow-matching setting, yielding a preference objective expressible directly through the model’s denoising loss. Crucially, we use the frozen reference model itself — in policy and random-action-proposer modes — to synthesize the dispreferred trajectories needed for the DPO contrast, eliminating the dependence on external reward labels or human ranking. The aligned model is trained with low-rank adapters so the play-acquired prior is preserved by construction. We outline the method, derive its objective, describe its concrete realization in the **dreamerV4-UWM** codebase, and propose an experimental program built on the existing **pushT** suite plus a longer-horizon manipulation task. **[TODO: numbers, ablations]**

1 Introduction

Joint world-action models trained on broad robotic “play” data offer an attractive prior for downstream control: they capture physical regularities and a wide span of behaviors, rather than collapsing onto a single task-specific policy. The dreamerV4-UWM line of work pushes this further by exposing *independent* per-frame noise levels for the state and action streams of a flow-matching denoiser, so that one set of weights can be inference-time configured into three roles: world model, policy, or random-action proposer.

Two questions follow naturally from such a system:

1. Once a play prior is in place, how do we shift it toward expert behavior on a target task using a small demonstration set, *without* catastrophically forgetting the breadth that motivated play pretraining in the first place?
2. What is the right post-training objective, given that the model’s policy is implicit in a flow-matching denoiser whose action log-likelihood is intractable in closed form?

A natural answer to (i) is preference optimization — demonstrations are by construction preferred over rollouts of the broad prior — and the natural answer to (ii) is the Diffusion-DPO formulation, which expresses the preference bound entirely in terms of the denoising loss. These

two ingredients combine in a way that uses the model’s distinguishing capabilities: *the world model itself supplies the negative trajectories that the demonstrations are contrasted against*. The dual-channel architecture is no longer just an inference-time switch but a structural component of the alignment objective.

Contributions.

- A formulation of trajectory-level DPO for joint flow-matching world-action models with independent state/action noise levels, derived as a specialization of the Diffusion-DPO bound (§3.1).
- A self-contained source of dispreferred trajectories generated by the frozen reference model in its policy and random-action-proposer modes, removing the need for external reward labels (§3.2).
- A concrete instantiation in the **dreamerv4-UWM** codebase using LoRA adapters on the temporal blocks of the denoiser, with bit-equivalent fallback when alignment is disabled (§4).
- A proposed experimental program that isolates each ingredient against ablation baselines, including a critical “what does this do to modes 1, 2, and 3?” regression check that we treat as a first-class outcome (§5).

2 Background

2.1 Joint flow-matching world-action transformer

The backbone (`dreamerv4uwm/models/dynamics.py:DreamerV4Denoiser`) is a causal transformer over a per-frame token sequence

$$\text{Frame}_t = [\mathbf{Z}_t, \mathbf{R}_t, \text{IC}_t, \text{AC}_t, \mathbf{A}_t], \quad (1)$$

where $\mathbf{Z}_t$ are the latent tokens of frame t produced by a frozen tokenizer, $\mathbf{R}_t$ is a small register, IC_t and AC_t are write-only *control tokens* encoding the per-frame noise level for state and action respectively (and any shortcut step index), and $\mathbf{A}_t$ is the (possibly noisy) action token. Spatial attention is masked so the controls are read-only conditioning, and temporal attention is causal across frames.

Noise convention. We use the project’s convention $\tau \in [0, 1]$ where $\tau = 1$ is clean data and $\tau = 0$ is pure noise, with linear forward mixing

$$x_\tau = \tau x_{\text{clean}} + (1 - \tau) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2)$$

The denoiser is trained as an x -predictor: given x_τ and τ it regresses x_{clean} . Independent per-frame noise levels τ_t^z for the state stream and τ_t^a for the action stream are encoded into the corresponding control tokens.

Three inference modes. A single trained denoiser is used in three roles by choosing the per-frame (τ_t^z, τ_t^a) pattern:

- **World model (WM):** clean context (states and actions), noisy horizon states, clean horizon actions $\Rightarrow$ predict horizon states given a candidate action sequence.

- **Policy (π):** clean context (states and actions), noisy horizon *both* streams $\Rightarrow$ jointly generate states and actions on the horizon.
- **Random-action proposer (RAP):** *noisy* context (states and actions), noisy horizon *both* streams $\Rightarrow$ broad joint generation that covers a search space for downstream planners.

Flow-matching loss. For a clean trajectory $\xi = (z_{1:T}, a_{1:T})$ and per-frame noise levels $\tau = (\tau_{1:T}^z, \tau_{1:T}^a)$, the per-sample denoising loss is

$$\begin{aligned} \ell_{\theta}(\xi, \tau, \epsilon) = & \sum_{t=1}^T \left[w(\tau_t^z) \|D_{\theta}^z(z_{1:T}, a_{1:T}, \tau)_t - z_t\|^2 \right. \\ & \left. + w(\tau_t^a) \|D_{\theta}^a(z_{1:T}, a_{1:T}, \tau)_t - a_t\|^2 \right], \end{aligned} \quad (3)$$

where $z^{(\tau)}, a^{(\tau)}$ are the streams mixed with independent Gaussian noise ϵ at the prescribed levels and $w(\cdot)$ is a noise-level loss weight (uniform or ramp; §5.6). The expected loss is $\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\xi, \tau, \epsilon} [\ell_{\theta}(\xi, \tau, \epsilon)]$.

In the existing codebase, the joint $(\tau_{1:T}^z, \tau_{1:T}^a)$ pattern is sampled per minibatch from a mixture indexed by mode — `policy`, `wm`, `id`, `video`, `forcing` — with mixing weights set by `train.mode_weights`. We will reuse this sampler verbatim, but at *evaluation* time within the alignment loop (§3.1).

2.2 Direct Preference Optimization

Given a reference policy π_{ref} and a dataset of preference pairs $(\xi^+, \xi^-) \sim \mathcal{D}$, DPO [1] optimizes

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\xi^+, \xi^-) \sim \mathcal{D}} \left[\log \sigma(\beta [r_{\theta}(\xi^+) - r_{\theta}(\xi^-)]) \right], \quad (4)$$

where the implicit reward is $r_{\theta}(\xi) = \log \pi_{\theta}(\xi) - \log \pi_{\text{ref}}(\xi)$ and $\beta > 0$ controls deviation from the reference. The objective is closed-form (no value network), but it presupposes that $\log \pi_{\theta}(\xi)$ is tractable.

2.3 Diffusion-DPO

For diffusion / flow-matching generative models, the trajectory log-likelihood is intractable but admits a variational upper bound expressible through the denoising loss. *Diffusion-DPO* [2] substitutes this bound into Eq. (4): up to a positive constant,

$$\log \pi_{\theta}(\xi) - \log \pi_{\text{ref}}(\xi) \geq -\mathbb{E}_{\tau, \epsilon} \left[w(\tau) (\|D_{\theta}(\xi_{\tau}, \tau) - \xi\|^2 - \|D_{\text{ref}}(\xi_{\tau}, \tau) - \xi\|^2) \right] + C, \quad (5)$$

which after substitution into Eq. (4) gives a fully tractable preference loss. We adapt this bound to the joint state-action setting in §3.1.

3 Method

3.1 Flow-DPO over joint state-action trajectories

Define the per-trajectory *advantage* of the aligned model over the reference under shared noise samples (τ, ϵ) :

$$\Delta_{\theta}(\xi; \tau, \epsilon) := \ell_{\text{ref}}(\xi, \tau, \epsilon) - \ell_{\theta}(\xi, \tau, \epsilon), \quad (6)$$

where ℓ_θ is the per-sample flow-matching loss in Eq. (3). By Eq. (5), $\mathbb{E}[\Delta_\theta(\xi; \cdot)]$ is a lower bound on $\log \pi_\theta(\xi)/\pi_{\text{ref}}(\xi)$ up to a constant that cancels in differences.

The **Flow-DPO loss** for a preference pair (ξ^+, ξ^-) is

$$\mathcal{L}_{\text{FDPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \left[\Delta_\theta(\xi^+; \boldsymbol{\tau}^+, \epsilon^+) - \Delta_\theta(\xi^-; \boldsymbol{\tau}^-, \epsilon^-)\right]\right)\right], \quad (7)$$

with the outer expectation taken over $(\xi^+, \xi^-) \sim \mathcal{D}$ and the noise variables. Two practical comments:

Shared noise across the pair. Following Wallace et al. [2], we use the *same* $(\boldsymbol{\tau}, \epsilon)$ for ξ^+ and ξ^- within a pair. This is a strong variance-reduction trick: the constants in Eq. (5) match exactly between positive and negative, leaving only the model-dependent terms in the difference.

Mode sampler at evaluation time. We sample $(\boldsymbol{\tau}^z, \boldsymbol{\tau}^a)$ from the project’s existing policy-mode schedule (clean context, sampled- τ horizon for both streams) when scoring a preference pair. This ensures the DPO gradient sees the policy regime — the regime where ξ^+ vs. ξ^- actually matters — and does not leak weight onto the world-model regime (which we do not want to perturb). Other modes can be ablated.

Per-frame loss weighting. The loss weight $w(\tau)$ is the **ramp** or **uniform** schedule from `dreamerv4uwm/loss.py`. Note (§5.6) that **ramp**, $w(\tau) = 0.9\tau + 0.1$, *down-weights* the noisy regime under the project’s $\tau=1 \Rightarrow$ clean convention. We default to **uniform** for FDPO since the preference signal at moderate-to-high noise carries the structural information about coarse trajectory shape.

Why this is honest about the architecture. The implicit policy in Eq. (7) is the actual flow-matching denoiser D_θ : every gradient step adjusts the score field that the model uses to generate actions, not a parametric Gaussian projection. The dual-channel structure — separate state and action terms in ℓ_θ — enters the loss linearly, so the alignment objective decomposes into “align state predictions on ξ^+ vs. ξ^- ” plus “align action predictions on ξ^+ vs. ξ^- ”. We can mute either side via a coefficient if needed (§5.4).

3.2 Where do the negatives come from?

The most fragile part of any trajectory-level DPO setup is the source of ξ^- . Random sequences make the contrast trivial; on-policy rollouts of π_θ require simulation; and human ranking is expensive. We exploit the dual-channel architecture to generate negatives from the *frozen reference model itself*, in two complementary modes.

Reference policy negatives (ξ_π^-). Given an initial state s_0 matched to a demonstration, we run D_{ref} in its *policy* mode (clean context, jointly denoise horizon states and actions) using its standard sampling procedure. The resulting ξ_π^- is what the broad play prior would produce from s_0 . Demonstrations ξ^+ are by construction preferred to ξ_π^- : the demonstration is the expert sequence on this initial condition, while ξ_π^- is a plausible play-prior continuation that may or may not be expert. The DPO contrast pushes D_θ to assign higher implicit likelihood to demonstrations than to the prior on the same initial conditions.

RAP-mode negatives (ξ_{RAP}^-). Generated by D_{ref} in random-action-proposer mode (noisy context, joint generation), ξ_{RAP}^- samples are explicitly designed to be diverse and exploratory. They form a strong notion of “physically plausible but task-uninformed” trajectories. Including them in $\mathcal{D}^-$ shapes D_θ to assign implicit likelihood mass that is concentrated near demonstrations even relative to the prior’s broad support.

Multi-source negative mixture. We use a mixture

$$\mathcal{D}^- = \alpha_\pi \mathcal{D}_\pi^- \cup \alpha_{\text{RAP}} \mathcal{D}_{\text{RAP}}^- \quad (8)$$

with mixing weights $\alpha_\pi + \alpha_{\text{RAP}} = 1$. The α ratio is treated as a hyperparameter (default 0.5/0.5). Because both sources come from the *frozen* D_{ref} , the negative distribution does not drift during training, eliminating one of the standard pathologies of online DPO.

Why the world-action backbone is load-bearing here. This is the key methodological point: the same network that we are trying to align is also the one that supplies the negatives. The dual-channel architecture is not decorative — without RAP mode we lose the “broad off-policy negatives” arm, and without WM-quality joint generation we lose the “in-distribution prior negatives” arm. A purely policy-only baseline cannot replicate this without an external simulator.

3.3 Segment-level preferences

Trajectory-level Gaussian-style log-likelihoods are dominated by the longest streams; in our setting, the worst-positioned step of ξ^+ can flip the sign of the per-pair DPO term and inject high variance. We use *segment-level* preferences: for each demonstration episode, we slide a window of length $L_s \leq T_{\text{ctx}}$ across the episode and form preference pairs at each window position $(\xi_{[t, t+L_s]}^+, \xi_{[t, t+L_s]}^-)$ where the negative is generated by D_{ref} from the demonstration’s own context $[1, t-1]$. This:

1. amplifies the number of supervisable preferences per demonstration roughly by T_{episode}/L_s ;
2. avoids drift accumulation in the negative trajectory by always conditioning on the true demonstration history up to the segment start;
3. matches the temporal scale at which the demonstration vs. prior comparison is well-defined — whole-episode comparisons confound many local sub-decisions.

We treat the segment length L_s as a primary ablation knob (§5.4).

3.4 LoRA on the joint backbone

The aligned model D_θ is the reference D_{ref} plus low-rank adapters on a chosen subset of layers. Concretely, for any linear projection $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ in the targeted set,

$$W_\theta = W_{\text{ref}} + \Delta W, \quad \Delta W = B A, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}), \quad (9)$$

with A Gaussian-initialized and B initialized to zero so that $D_\theta = D_{\text{ref}}$ at step zero. This makes the DPO term in Eq. (7) exactly zero at initialization (the advantage difference vanishes), which gives a clean optimization start with no spurious early gradient.

We target the temporal blocks’ $\{Q, K, V, O\}$ projections (the locus of multi-step decision-making) and leave the spatial blocks frozen by default. Spatial-LoRA and full-fine-tune are ablations.

3.5 The full objective

We optimize a convex combination of the DPO term and a small flow-matching anchor on the demonstration set:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{FDPO}}(\theta) + \lambda_{\text{anchor}} \mathbb{E}_{\xi^+ \sim \mathcal{D}^+} [\ell_{\theta}(\xi^+, \tau, \epsilon)]. \quad (10)$$

The anchor term is the same per-sample loss the backbone was trained with, restricted to demonstrations, and serves to keep the absolute scale of D_{θ} 's flow predictions calibrated to the data — without it, DPO tends to inflate D_{θ} 's loss on ξ^- by raising loss everywhere, which is a known failure mode [4, 5]. We default to small $\lambda_{\text{anchor}} \in [10^{-2}, 10^{-1}]$ and ablate.

4 Implementation in dreamerV4-UWM

This section makes the method concrete against the existing codebase. Code paths refer to current `main`.

4.1 Stage 0: pretrained backbone

We assume a checkpoint θ_{ref} from `train_dynamics_uwm.py` on play data, ideally one of the recipes that already handles all three modes acceptably (e.g. the `progressive-forcing-action-video-wm-image` family in `docs/experiments.md`). The state of the backbone with respect to mode 3 is part of the experimental story — a weaker mode-3 reference makes the RAP-mode negatives noisier but also makes the alignment-induced improvement easier to detect.

4.2 Stage 1: precompute negatives

We add a one-time precomputation step that, for every demonstration window $\xi_{[t, t+L_s]}^+$, samples K negatives from D_{ref} in policy and RAP modes using the existing flow-matching sampler (`dreamerv4uwm/sampling.py[TODO: verify path]`). Negatives are cached as a sibling shard layout (`shard_XXXX_neg.h5`) keyed on the same window indices, with payload (`image_neg`, `action_neg`, `source_id`, `neg_id`). This keeps the alignment training loop offline-style, no on-policy sampling overhead.

This is computationally substantial but bounded: $|D^+| \cdot K \cdot 2$ rollouts per epoch-equivalent, with $K \in \{2, 4, 8\}$ in our setup. Each rollout is one forward pass of D_{ref} at the chosen $T_{\text{ctx}} + L_s$ horizon at modest sampling steps.

4.3 Stage 2: alignment training script

We add a new entry point `scripts/train_align_fdpo.py` parallel to `train_dynamics_uwm.py`. The script:

1. Constructs D_{ref} from a checkpoint, freezes it, places it in `eval + no_grad`.
2. Constructs D_{θ} as a copy of D_{ref} wrapped with LoRA adapters (PEFT-style, with $\Delta W = 0$ at init).
3. For each step:
 - (a) Draws a paired batch (ξ^+, ξ^-) from a new `PreferenceShardedDataset` that joins the demonstration shards with the negative shards built in §4.2.

- (b) Samples one (τ, ϵ) pair via `UWMForwardProcess` restricted to the `policy` mode, applied identically to ξ^+ and ξ^- .
 - (c) Runs both D_θ and D_{ref} in a single batched forward (no gradient through D_{ref}).
 - (d) Computes $\ell_\theta(\xi^\pm, \tau, \epsilon)$ and $\ell_{\text{ref}}(\xi^\pm, \tau, \epsilon)$ via a new `compute_per_sample_uwm_loss` that wraps `compute_uwm_loss` without reduction.
 - (e) Forms the FDPO term per Eq. (7) and the anchor term per Eq. (10); sums and back-propagates only into the LoRA parameters.
4. Periodically writes evaluation rollouts (mode 1, mode 2, mode 3) to verify no regression — see §5.

4.4 Code points to extend

- `dreamerv4uwm/loss.py`: add `compute_per_sample_uwm_loss(...)` returning $(B,)$ -shaped per-sample loss; refactor `compute_uwm_loss` to call into it. Existing call sites untouched.
- `dreamerv4uwm/models/dynamics.py`: add a thin `LoRAWrappedDenoiser` that takes a `DreamerV4Denoiser` and a target-module list and produces a wrapper exposing only the LoRA parameters as trainable. No structural change to the denoiser itself.
- `dreamerv4uwm/datasets.py`: `PreferenceShardedDataset` that pairs (ξ^+, ξ^-) windows and shares the existing windowing math.
- `scripts/config/align/`: new config namespace mirroring `dynamics/` (e.g. `align/pushT-fdpo.yaml`) with β , λ_{anchor} , L_s , LoRA rank, mode-mixture weights $\alpha_\pi/\alpha_{\text{RAP}}$, and the source D_{ref} checkpoint path.
- `hpc/slurms/align/pushT/*.slurm`: cloned from the dynamics slurms, swapping the entry point.

By construction, with $\beta = 0$ and $\lambda_{\text{anchor}} = 1$ and LoRA rank r but $\xi^- = \xi^+$, the script reduces to demonstration BC on the LoRA parameters — providing a built-in sanity check.

4.5 Compatibility with the shortcut variant

The shortcut bootstrap path (`train_dynamics_uwm_with_shortcut.py`) trains the same denoiser with an additional teacher-bootstrap head over a dyadic step grid. The FDPO loss is orthogonal: it operates on the flow branch (`step_index = max_pow2`) per Eq. (3). We therefore restrict the alignment loop to the flow branch and leave the bootstrap branch untouched, both at training and at inference. A future variant could attempt FDPO over the bootstrapped denoiser; we leave it out of scope.

5 Proposed experiments

5.1 Tasks and datasets

We propose three task settings:

- **PushT (dreamerV4-UWM’s standard)**. Short-horizon planar manipulation. Already exercised by the existing dynamics recipes; suitable for the first round of alignment ablations because the play-vs-demo gap is well-characterized.

- **LeRobot tabletop manipulation suite (multi-task).** The `lewm-cubes` / `lewm-pushT` family already supported by the codebase. Tests whether the play prior, when concentrated by FDPO toward a task subset, degrades on the other tasks — the central retention claim.
- **Long-horizon manipulation (target).** [TODO: Pick a concrete long-horizon task from the LeRobot or RLBench corpus; this is the headline-result task and the one that motivates why a play prior should help.]

5.2 Claims to test

We aim for a small set of sharp claims, each of which has a corresponding ablation:

- **C1.** Alignment via FDPO improves task success over BC on the same demonstrations.
- **C2.** World-model-generated negatives outperform random-action negatives.
- **C3.** LoRA on temporal blocks preserves play-acquired competence; full fine-tune does not.
- **C4.** Segment-level preferences outperform episode-level.
- **C5.** Aligning the joint state-action backbone outperforms aligning a policy-only head.

5.3 Baselines

Baseline	What it tests
BC on demos only (no play pretrain)	Value of the play prior.
Play-pretrained D_{ref} + BC LoRA	Value of preference signal beyond NLL.
Play-pretrained D_{ref} + full FT BC on demos	Value of LoRA / KL-style anchoring.
Policy-only DPO (frozen backbone, MLP head)	Value of aligning the joint backbone.
FDPO with random-action negatives	Value of WM-mode negatives.
FDPO with on-policy aligned-model negatives	Value of frozen-source negatives.
FDPO with WM+RAP negatives (ours)	Headline.

Table 1: Baseline grid. Each row isolates one of C1–C5.

5.4 Ablations

- $\beta \in \{0.01, 0.1, 1.0, 10.0\}$.
- LoRA rank $r \in \{4, 8, 16, 32\}$, target set $\in \{\text{temporal-only}, \text{spatial-only}, \text{both}\}$.
- Anchor weight $\lambda_{\text{anchor}} \in \{0, 0.01, 0.1, 1.0\}$.
- Negative mixture $(\alpha_{\pi}, \alpha_{\text{RAP}}) \in \{(1, 0), (0.5, 0.5), (0, 1)\}$.
- Segment length L_s vs. episode length.
- Per-stream FDPO mask: state-only contrast, action-only contrast, both.
- Loss weighting $w(\tau)$: **uniform** vs. **ramp**.

5.5 The mandatory regression check

We adopt as a first-class outcome the following “modes 1, 2, 3” regression evaluation, run after every alignment training run:

- **Mode 1 (WM):** state prediction RMSE on a held-out episode set under clean-context, clean-action, sampled- τ^z horizon. Compared to D_{ref} .
- **Mode 2 (π):** task success and trajectory return when sampling jointly from a clean context.
- **Mode 3 (RAP):** entropy and coverage diagnostics of joint state-action samples from noisy context (existing diagnostics in `docs/experiments.md`).

A run that improves Mode 2 task success by less than the regression in Mode 1 RMSE or Mode 3 coverage is reported as a wash, not a win. This is the project-internal contract from `CLAUDE.md` and we treat it as a publishable property of the method.

5.6 Loss weighting nuance

Because the project’s **ramp** weighting $w(\tau) = 0.9\tau + 0.1$ down-weights the noisy regime under the inverted noise convention, FDPO with **ramp** would put the bulk of its preference signal at $\tau \approx 1$ (nearly clean) — where the contrast between ξ^+ and ξ^- is smallest because the denoiser has very little to “do”. We therefore default to **uniform** and treat **ramp** as an ablation to characterize the failure mode, not as the headline.

6 Risks and mitigations

Reward-hacking by raising absolute loss. A standard DPO failure mode is to make ξ^- much worse without making ξ^+ better, since only the difference enters $\sigma(\cdot)$. The anchor term in Eq. (10) explicitly counteracts this by penalizing high absolute loss on demonstrations. We will monitor the absolute $\ell_\theta(\xi^+)$ over training; a sustained rise is a red flag.

Negative collapse. If D_{ref} ’s policy-mode samples are already close to ξ^+ for some initial conditions, the DPO contrast is near-zero and provides no gradient. We mitigate by (a) including RAP-mode negatives, which are deliberately broad, and (b) re-sampling negatives periodically (every N epochs) to refresh the pool.

Distributional shift in the WM channel. If FDPO drives the action-stream LoRA hard, the implicit conditioning on actions inside the WM mode could degrade rollout fidelity. Two safeguards: keep LoRA rank low; report Mode 1 RMSE as a hard regression gate (§5.5).

Compute cost of negatives. Generating K rollouts per window for the entire dataset is non-trivial but is a one-time precomputation per D_{ref} . For a refreshing schedule, we cap to one refresh per $\sim 10\text{k}$ optimization steps.

Episode boundary effects in segment preferences. Sliding windows at the end of an episode hit the boundary; we follow the existing dataset windowing convention (`dreamerv4uwm/datasets.py`) and drop windows that span past the episode end.

7 Related work (sketch)

[**TODO: Tighten with citations during paper writing.**] Closest work: Diffusion-DPO [2], which derives the bound we adapt; world-model policy optimization (WM-PO, Crossway diffuser) which uses world models to score actions but with explicit reward objectives; LoRA-based post-training [3]; preference optimization for action diffusers in robotics. Distinct from prior robotics DPO in (a) joint state-action flow-matching backbone, (b) WM-self-generated negatives, (c) explicit retention guarantee on the world-model and random-action-proposer modes.

8 Discussion

The aspect of this proposal we are most uncertain about is whether the *magnitude* of the FDPO gradient through the temporal LoRA is large enough to meaningfully reshape the action distribution at sampling time, given that the implicit policy is realized through dozens of denoising steps. Diffusion-DPO works in image generation, where the analogous concern is real but empirically tractable; the robotics setting differs in that we care about the conditional density at very specific control-relevant frequencies, not perceptual statistics. The headline experiment to run before scaling up is therefore a sanity check that a small FDPO LoRA on a frozen D_{ref} can recover the BC solution on a target task — if it cannot, the bound is too loose at our scales and a different formulation is warranted.

The aspect we are most confident about is the negatives story: “use the world-action model in its three inference modes to manufacture preference data” is exactly the use case the dual-channel architecture was designed to enable, even if the original motivation was downstream planning rather than alignment.

References

- [1] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. Manning, and C. Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS*, 2023.
- [2] B. Wallace, M. Dang, R. Rafailov, L. Zhou, A. Lou, S. Purushwalkam, S. Ermon, C. Xiong, S. Joty, and N. Naik. Diffusion Model Alignment Using Direct Preference Optimization. *CVPR*, 2024.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [4] M. G. Azar et al. A General Theoretical Paradigm to Understand Learning from Human Preferences. *AISTATS*, 2024.
- [5] J. Xu et al. Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study. *arXiv:2404.10719*, 2024.

[**TODO: Add: Dreamer-V4; the dreamerV4-UWM tech report once written; flow-matching foundations (Lipman et al.); WM-PO; relevant robotics DPO work.**]